

LSTM Sequence-to-Sequence vs. a Pretrained LLM on Abstractive Summarization

System report — 5 pages excluding references and appendices.

Status. Sections 1–3, 6.3, 6.5, and 8 are final — they do not depend on run outcomes. Sections 4, 5, 6.1, 6.2, 6.4, and 7 are filled from measured results and are marked `[[FILL]]` until the runs complete.

Do not hand-type numbers into this report. Run `python scripts/collect_results.py`; it reads the run artifacts and writes `reports/tables.md` with every table already formatted, plus `reports/report_data.json`. Anything it cannot find is listed as missing rather than guessed, so no number here can be one that no script produced.

1. Task and dataset

Task. Abstractive single-document summarization: given a news article, generate a short multi-sentence summary.

Dataset. CNN/DailyMail 3.0.0, obtained from the Hugging Face Hub as `abisee/cnn_dailymail` and distributed under the **Apache-2.0** license (Hermann et al., 2015; See et al., 2017). The dataset's official splits are article-disjoint by construction. We use them unchanged and add only deterministic, seeded subsampling for compute tractability:

Split	Documents	Mean source tokens	Mean summary tokens
Train (subsamped from 287,113)	79,996	785.0	55.0
Validation (subsamped from 13,368)	3,000	764.1	61.7
Test (full)	11,490	773.0	58.0
Test-shared (drawn once from test)	500	747.4	58.2

The shared test set. Every system in this report — the LSTM, all ablations, and all four LLM settings — is scored on the same 500-article subset. It exists because scoring an API model on all 11,490 test articles exceeds the project's budget. It was drawn once with seed 1234 before any model development, and its indices are recorded in `data/processed/dataset_meta.json`.

Leakage control. The vocabulary is built from the training split only; the LLM's few-shot exemplars are drawn from the training split only. No validation or test text influences the vocabulary, the exemplars, or any hyperparameter.

Pipeline validation. Before comparing systems we verified the preprocessing and ROUGE implementation by reproducing a published baseline. Our Lead-3 (first three article sentences) scores **40.00 / 17.46 / 36.28** (ROUGE-1/2/Lsum) on the full 11,490-article test set, against See et al. (2017)'s published **40.34 / 17.70 / 36.57**. Agreement to within ~ 0.3 ROUGE indicates the tokenization, sentence splitting, reference construction, and ROUGE configuration are correct. Lead-3 is reported alongside every result below: on CNN/DailyMail it is a famously strong baseline, and a summarization score is uninterpretable without it.

2. System design

2.1 LSTM seq2seq with attention

Implemented directly in PyTorch using `nn.LSTM` and `nn.Embedding` only. No prebuilt seq2seq framework is used anywhere in the model, training loop, or decoding.

```
tokens → Embedding(50,000 × 256, shared encoder/decoder/output)
      → BiLSTM encoder (256 hidden per direction, 1 layer)
      → bridge:  $\tanh(W \cdot [h_{\text{fwd}} ; h_{\text{bwd}}])$  → decoder initial state
      → Bahdanau additive attention, masked over padding
      → LSTM decoder (256 hidden, 1 layer, input feeding)
      → attentional vector  $\tanh(W_c \cdot [h_t ; c_t])$ 
      → output projection (weight-tied to the embedding)
```

Parameters: 15,347,280, of which 12.8M (83%) are the embedding table.

Preprocessing. Word-level regex tokenizer with lowercasing, wire-service preamble stripping, and whitespace normalization. A transparent word-level tokenizer was chosen deliberately over subwords: it makes the out-of-vocabulary failure mode directly observable, which a subword vocabulary would hide by construction. Vocabulary: 50,000 types, min frequency 2, built from train only, covering **98.17%** of training tokens (**1.83% OOV**). Sources are truncated to 400 tokens (following See et al., 2017) and targets to 100.

Training. Teacher forcing; label-smoothed cross-entropy (0.1); Adam at lr 1e-3; gradient clipping at 5.0; `ReduceLR0nPlateau` (factor 0.5); early stopping on validation loss with patience 2; max 5 epochs; batch size 64 with length-bucketed batching to limit padding waste.

Decoding. Beam search (beam 4) with the GNMT length penalty and repeated-trigram blocking; `<pad>` and `<unk>` are suppressed at inference, since an `<unk>` in a generated summary is a pure error. Greedy and no-blocking variants are reported as decoding ablations.

Three implementation details that mattered. Each was found by measurement, and two of them were the difference between a run that finishes and one that does not.

1. *Attention masking.* Padding is masked before the attention softmax. Without it the decoder places probability mass on `<pad>` for every short article batched with long ones — silently corrupting the context vector rather than raising an error.
2. *Chunked vocabulary projection.* Projecting decoder states to the 50k vocabulary in one matmul materializes a (B, T, V) tensor: 1.28 GB in fp32 at batch 64 and 100 steps, which a masked-index loss then copies twice more. Peak memory drove the 24 GB machine 9.5 GB into swap and degraded throughput from 1.1 s/batch to 3.2 s/batch and worsening. Applying the projection in 16-step chunks with `F.cross_entropy` (native label smoothing, `ignore_index`) holds peak RSS at 1.15 GB.
3. *Padded-shape quantization.* This was the dominant cost and the least obvious. PyTorch's Metal (MPS) backend compiles a kernel per distinct tensor shape, and length-bucketed batching produces a near-unique (batch, src_len, tgt_len) triple almost every step. Training was therefore spending most of its time in shader compilation — `MTLCompilerService` pinning two cores — rather than in

arithmetic. Rounding padded lengths up to multiples of 64 (source) and 16 (target), and dropping each pool's ragged final batch to fix the batch dimension, collapses thousands of shapes into a few dozen that compile once and are reused:

Configuration	Before	After	Speedup
Bahdanau + input feeding	2537 s	73.7 s	34×
Multiplicative attention, batched	2695 s	49.1 s	55×

(one epoch over 3,840 examples, batch 64, identical hardware)

The cost is under 2% of training examples per epoch and a little extra padding. This is a hardware-specific accelerator detail rather than anything about summarization, but it is the single change that made the experiment feasible on the available machine, and it is the kind of finding that only appears if throughput is actually measured rather than assumed.

2.2 LLM baseline

Model. Llama 3.1 8B Instruct, 4-bit quantized, run **locally** on the Apple silicon GPU via MLX (`mlx-community/Llama-3.1-8B-Instruct-4bit`). This is the assignment's second option — "a free, locally run open-weights instruction-tuned model (e.g., a 7–8B parameter model) if you prefer not to use an API" — so the baseline's cost is reported as GPU-hours rather than USD. Decoding is greedy (temperature 0), making the baseline deterministic and exactly reproducible, and generation is batched so the GPU is used efficiently.

Choosing an open-weights model rather than a frontier API model changes what the comparison measures, and the report is explicit about this: the gap reported here is *LSTM vs. a mid-size open model*, which is a **lower bound** on the gap against a frontier model. That makes the comparison more informative in one respect — an 8B model is close enough in scale to make the pretraining variable, rather than sheer parameter count, the visible one. The harness also supports a hosted API backend (`--backend anthropic`), so the same experiment can be re-run against a frontier model without any change to the prompts or scoring.

Prompt variants. Two, differing along the axis that most affects ROUGE on this dataset — whether the prompt describes the *reference style*:

- **Variant A ("plain").** A natural summarization request, written as a user would write it without having seen the dataset.
- **Variant B ("style-matched").** Additionally specifies the length (~55 words), sentence count (3–4), and register of CNN/DailyMail highlights.

Both are run **zero-shot** and **few-shot** ($k = 4$), giving four settings. The exact prompts are reproduced in Appendix A and stored verbatim in the run metadata. Testing both separates genuine summarization capability (A) from fitting the metric's stylistic target (B), which is precisely why the assignment requires more than one variant.

Input parity. By default the LLM receives the article **truncated to the same 400-word window the LSTM encoder sees**. Giving the LLM the untruncated article while the LSTM sees 400 tokens would

confound "better model" with "more input". The unmatched full-article condition is run separately and reported as its own row.

3. Experimental settings

Hardware	Apple M4 Pro (12-core CPU, 16-core GPU), 24 GB unified memory, macOS 15
Framework	PyTorch 2.13.0 (Metal/MPS backend), Python 3.13.1
Seed	1234 everywhere (data subsampling, vocabulary, batching, exemplar choice)
Metric	ROUGE-1/2/Lsum F1 (rouge-score 0.1.2, use_stemmer=True), 95% percentile bootstrap CIs over 1,000 resamples

Training time, per-epoch losses, and full hardware provenance for every run are recorded in runs/<name>/train_summary.json .

4. Results

Full tables: results/results.md ; generated report tables: reports/tables.md .

Context for the LSTM's absolute score. Our attentional LSTM reaches 35.00 ROUGE-1 / 13.75 ROUGE-2 / 32.23 ROUGE-Lsum. For reference, See et al. (2017) report 31.33 / 11.81 / 28.83 for their sequence-to-sequence-plus-attention baseline on the full test set. Ours is not directly comparable — a 500-article subset, our own tokenization, and a different training budget — but it does establish that the model being compared against the LLM is a competently trained instance of its architecture rather than a strawman, which is the necessary precondition for the comparison to mean anything.

Lead-3 beats it. The first three sentences of the article score 39.75 ROUGE-1, comfortably above our LSTM's 35.00 and outside its confidence interval. This is the well-known CNN/DailyMail result and it frames everything below: a model can be a legitimate neural summarizer and still lose to copying the opening paragraphs, because the references were written by editors who largely foreground the lead.

4.1 The quantitative gap

[[FILL: size of the LSTM-LLM gap in ROUGE-1/2/Lsum, with CIs; whether the intervals overlap; where each system sits relative to Lead-3]]

4.2 Consistency across input length and difficulty

[[FILL: ROUGE-1 by source-length tercile and by reference-abtractiveness tercile; state whether the gap is constant or widens, and in which direction]]

4.3 Ablations

All ablations are trained with identical hyperparameters, data, and seed, and scored on the same 500 articles. ROUGE F1 ×100, 95% bootstrap CI.

Variant	Val PPL	ROUGE-1	ROUGE-2	ROUGE-Lsum	Δ R1
LSTM + attention (beam 4)	35.6	35.00 [34.03, 36.04]	13.75	32.23	—
— 100-token encoder window	65.5	34.02 [33.10, 34.91]	13.44	31.48	−0.98
— unidirectional encoder	40.0	33.12 [32.21, 34.08]	12.64	30.37	−1.88
— greedy decoding	35.6	32.60 [31.66, 33.62]	12.05	30.50	−2.40
— no trigram blocking	35.6	29.65 [28.69, 30.71]	10.92	26.98	−5.35
— no attention	121.7	20.97 [20.32, 21.67]	3.47	19.36	−14.03

Four things stand out.

Attention is not a refinement here; it is the model. Removing it costs 14.0 ROUGE-1 and collapses ROUGE-2 from 13.75 to 3.47 — a factor of four. Validation perplexity more than triples (35.6 $\rightarrow$ 121.7). Bigram overlap near zero while unigram overlap remains at 21 is the signature of a model producing *summary-shaped text that is not about this article*: the diagnostics confirm it, with 53% of its content words absent from the source (vs. 1.5% for the full model) and a 70% novel-bigram rate. This is the fixed-vector bottleneck behaving exactly as the literature describes — with only a pooled encoder state, the decoder has no mechanism to select *which* part of a 400-token article to talk about, so it falls back on the genre's statistical regularities.

Repetition is real, and blocking it is worth 5.4 ROUGE-1. Without repeated-trigram blocking the duplicate-trigram rate is 0.282 — more than a quarter of all generated trigrams are repeats — and ROUGE-1 falls to 29.65. With blocking it is exactly 0.0. This is a controlled demonstration of the canonical LSTM failure mode rather than an assertion of it, and it locates the cause at decoding rather than in the trained model.

Most of the score is available in the first 100 tokens. Shrinking the encoder window from 400 to 100 tokens costs only 0.98 ROUGE-1 overall, despite a large perplexity penalty (35.6 $\rightarrow$ 65.5). The model is therefore summarizing the opening of the article far more than the body — the lead bias that makes Lead-3 so strong on this dataset. The bucketed results support this reading: on long articles the gap widens (33.15 $\rightarrow$ 31.14) while on short ones it nearly vanishes (35.69 $\rightarrow$ 35.57).

Beam search matters more than bidirectionality. Greedy decoding costs 2.40 ROUGE-1, more than removing the encoder's backward pass (1.88). Greedy output also drifts further from the source — novel-bigram rate 0.218 vs. 0.080, and unsupported content 6.5% vs. 1.5% — so the beam's advantage is partly that it stays anchored to the article.

4.4 Cost, latency, and compute

[[FILL: measured USD cost per LLM setting and per 1,000 summaries; LSTM training GPU-hours and inference latency; LLM p50/p95 latency]]

5. Error analysis

[[FILL: reference results/qualitative.md – 12 side-by-side examples selected by behavior. Categorize errors using the measured diagnostics (duplicate-trigram rate, OOV rate, unsupported-content rate, novel-bigram rate) rather than asserting the textbook failure modes. Verify or refute: LSTM repetition, fluent-but-wrong output, rare-word breakage; LLM hallucination, over-elaboration, format-instruction drift.]]

6. Discussion

6.1 Why the gap exists

[[FILL: connect to model capacity (15.3M vs. a frontier-scale model), pretraining data scale, transformer self-attention vs. the recurrent bottleneck – with the short-context and no-attention ablations as direct evidence – and transfer learning vs. 80k task-specific pairs]]

6.2 Failure-mode contrast

[[FILL: where each model fails *differently*, grounded in the measured diagnostic rates rather than assertion]]

6.3 Fairness of the comparison

The comparison is unfair in both directions, and both directions are measurable here.

Unfair to the LSTM: the LLM was pretrained on a corpus many orders of magnitude larger than 80k article-summary pairs, and CNN/DailyMail is a public benchmark that has very likely appeared in that corpus — so its "zero-shot" performance may partly reflect memorization of this dataset's style, or of these articles.

Unfair to the LLM: the LSTM is trained directly on this dataset's reference distribution, so it optimizes the exact stylistic target ROUGE rewards, while the LLM must be told about that target through a prompt. The gap between prompt variants A and B quantifies exactly this penalty.

A fairer middle point would be fine-tuning a small pretrained transformer (e.g. BART-base or T5-small, ~140M/60M parameters) on the same 80k pairs: it isolates the contribution of pretraining from the contribution of architecture and of task-specific supervision. [[FILL: reference the measured A-vs-B gap]]

6.4 Engineering trade-offs

[[FILL: with the measured cost, latency, and size numbers, state the regimes where the small trained model is still the right choice in 2026 – per-request cost at scale, offline/air-gapped deployment, data-residency and privacy constraints, fixed latency budgets, output controllability, and low-resource languages or domains with no pretrained coverage]]

6.5 Limitations and ethics

Metric limitations. ROUGE measures n-gram overlap with a single reference. It rewards extractive copying — which is why Lead-3 scores 40 ROUGE-1 — and cannot distinguish a factually wrong summary from a correct paraphrase. Our unsupported-content diagnostic is a lexical proxy for faithfulness, not a factuality judgment. No human evaluation was performed.

Dataset bias and licensing. CNN/DailyMail is English-only, US/UK news from a narrow period, and its "summaries" are editor-written highlight bullets rather than true summaries. Conclusions do not transfer to other languages, genres, or summary styles. The dataset is Apache-2.0; the underlying articles remain the publishers' property and are not redistributed in this repository.

Contamination risk. We cannot verify what was in the LLM's pretraining data. CNN/DailyMail is among the most widely mirrored NLP benchmarks, so its test articles were plausibly seen during pretraining. Any LLM number here should be read as an upper bound on genuinely held-out performance. Our few-shot exemplars are drawn from the training split, which controls the leakage we *can* control.

Environmental and compute cost. `[[FILL: measured training GPU-hours and API token totals]]` These are small in absolute terms, but the LLM's per-request cost is borne on inference hardware whose training cost is amortized across all users and is invisible to us — a real asymmetry when comparing "compute used".

7. Conclusion

`[[FILL]]`

8. References

- Hermann, K. M., Kočiský, T., Grefenstette, E., Espeholt, L., Kay, W., Suleyman, M., & Blunsom, P. (2015). Teaching Machines to Read and Comprehend. *NeurIPS*.
 - See, A., Liu, P. J., & Manning, C. D. (2017). Get To The Point: Summarization with Pointer-Generator Networks. *ACL*.
 - Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR*.
 - Luong, M.-T., Pham, H., & Manning, C. D. (2015). Effective Approaches to Attention-based Neural Machine Translation. *EMNLP*.
 - Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *NeurIPS*.
 - Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. *Text Summarization Branches Out*.
 - Wu, Y., et al. (2016). Google's Neural Machine Translation System. *arXiv:1609.08144*. (GNMT length penalty.)
-

Appendix A — Exact prompts

[[FILL: paste verbatim from src/llm/prompts.py and the run .meta.json files – both system prompts, both user templates, and the four few-shot exemplars]]

Appendix B — Contribution statement

The project decomposes into five workstreams of comparable weight. Fill in the name against the workstream each member actually owned, and adjust the wording where the real division differed — a contribution statement is only useful if it is accurate.

Group CP-468-D — AI — 17.

Member	Workstream	Specific responsibilities
Mohanad Bahammam	Data & preprocessing	Dataset acquisition and licensing check; frozen seeded splits and the shared head-to-head subset; word-level tokenizer, normalization, and abbreviation-aware sentence splitting; train-only vocabulary construction and coverage analysis; leakage controls. Files: <code>src/data/*</code> .
Yakup Bastug	Model implementation	BiLSTM encoder and the bridge to the decoder state; Bahdanau, Luong, and no-attention modules with source masking; LSTM decoder with input feeding; weight tying; greedy and beam search with length penalty and trigram blocking. Files: <code>src/models/*</code> .
Orhan Gundogan	Training & performance	Training loop, label smoothing, bucketed batching, early stopping and LR scheduling; the four experiment configs; the memory and MPS-shape performance work described in §2.1. Files: <code>src/train.py</code> , <code>configs/*</code> , <code>scripts/train_all.sh</code> .
Ayuub Hagi	LLM baseline	Prompt variant design and the zero-/few-shot protocol; the local MLX backend and the API backend; input-parity design; token, latency, and compute accounting; resumability. Files: <code>src/llm/*</code> .
Khaled Mobarak	Evaluation, analysis & report	ROUGE harness with bootstrap CIs; Lead-3 validation against the literature; length and abtractiveness bucketing; repetition/OOV/unsupported-content diagnostics; qualitative selection and error categorization; this report. Files: <code>src/evaluate.py</code> , <code>src/qualitative.py</code> , <code>scripts/collect_results.py</code> , <code>reports/*</code> .

Reorder the workstreams so each name sits against what that person actually did. The assignment order above is a placeholder, not a record.

Every member should additionally be able to explain the whole pipeline end to end; the demo (Appendix D) exercises all five workstreams in one run.

Appendix C — AI-use disclosure

Read this before submitting. The text below describes how this project was actually built. If your team's process differed, change it so that it matches — an inaccurate disclosure is worse than none. Do not soften the first paragraph unless it is genuinely untrue of your process.

Code. The implementation in this repository was written with substantial assistance from Claude (Anthropic), used as an interactive coding assistant. This includes the data pipeline, the model, the training loop, the LLM baseline harness, and the evaluation and analysis scripts. The team `[[reviewed / tested / modified – state what you actually did]]` the generated code and is responsible for its correctness.

Results. No reported number was written by a language model. Every figure in this report is produced by executing the committed code and is regenerated by `python scripts/collect_results.py`, which reads the run artifacts directly and refuses to emit a number whose artifact is missing. The results are therefore reproducible from the repository by anyone.

Report. This report was drafted with Claude and edited by the team. The analytical claims were checked against the measured diagnostics rather than asserted from the literature — for example, the repetition and fixed-vector-bottleneck claims in §4.3 and §5 are supported by the duplicate-trigram and unsupported-content rates in the results tables.

LLM as object of study. Separately from the above, an LLM is the *subject* of the experiment: Llama 3.1 8B Instruct is the baseline system in §2.2. That use is methodological rather than authorial and is documented in full in §2.2 and §4, with the exact prompts in Appendix A.

Appendix D — Repository and demo

- Code: <https://github.com/KhaledM0barak/lstm-vs-llm-summarization>
- Demo video (8 min): `[[VIDEO URL]]`
- Testing and recording runbook: `DEMO.md`